

内存泄漏(memory leak):

没有撤消动态变量,而是把指向它的指针变量指向了别处或指向它的指针变量的生存期结束了,这时,这个动态变量虽然存在,但无法访问它,从而浪费空间。

例如:

```
int x,*p;
p=new int[10];//动态数组
P=&X;//之后,上面的动态数组就访问不到了!
//造成内存空间的丢失(内存泄露)
```

悬浮指针(dangling pointer):

用 delete 或 free 撤消动态变量后,指向它的指针就无效了(C++编译程序一般不会把指向它的指针变量的值赋为 0),这时该指针可能指向一个无效空间。

动态变量的应用 1——可扩充的数组

对于具有线性结构的复合数据,在程序中通常用数组来表示。

由于数组在定义时,元素个数必须是固定大小。当元素个数到运行时才能确定,下面的做法可行吗?

```
int n;
cin>>n;/数的个数
int a[n];//?
for (int i=0;i<n;i++)cin >>a[i];
```

C 语言的新标准允许在程序运行时指定数组元素个数,但是:

- (1)一旦指定后不能再改变!(上面数组 a 的大小不会随 n 变化)
- (2)不是每个 C++编译器都支持!

数组元素的插入/删除操作

a 为一个一维数组(假设已有元素个数为 n)。

删除数组 a 中位置 m 处的元素,赋给变量 x。

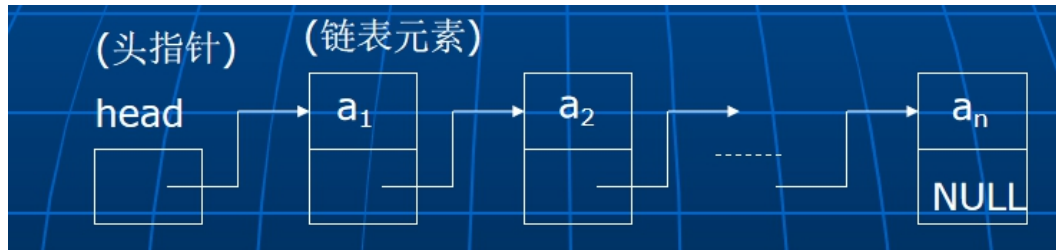
在数组 a 中位置 m 处插入一个元素 x。

动态变量的应用 2——链表

链表用于表示由数量不定的同类型元素所构成的具有线性结构的复合数据。

与动态数组不同:

- (1)链表的每个元素都是一个动态变量,并且是逐个生成的,因此扩充容易。
- (2)链表中的每一个元素除了本身的数据外,还包含一个(或多个)指针,它(们)指向链表中下一个(和其它)元素的内存位置,因此,各元素在内存中不必存放在连续的空间内。
- (3)需要有一个指针(头指针)指向链表的第一个元素,要访问链表中的某个元素,一般要从第一个元素开始遍历(通过头指针)。
- (4)在链表中插入/删除一个元素时一般只影响它前后两个元素。



单链表的每个元素（又称结点或节点）只包含一个指向下一个元素的指针。结点类型可定义如下：

```
struct Node //结点的类型定义
{
    int content; //存储结点的数据
    Node *next; //存储下一个结点的地址
};
```

另外需要一个头指针指向第一个结点，定义如下：

```
Node *head=NULL; //头指针变量定义，初始状态下
                //为空值。NULL 在 cstdio 中定义为 0
```

链表与数组的结合

数组与链表在表示线性元素构成的数据时各有特点：

数组访问元素效率高，但空间扩充困难，插入/删除效率低。

链表空间扩充容易，插入/删除效率高，但元素访问效率低。

可以把它们结合起来使用：

整体是链表，链表的每个结点中用数组存储多个元素。例如，结点的类型可以是：

```
struct Node //结点的类型定义
{
    int content[20]; //最多存储 20 个元素的数据
    int num; //本结点中元素的实际个数
    Node *next; //存储下一个结点的地址
};
```

访问元素时先找到元素所在的结点，在结点内部可用下标访问。

增加/删除元素在某结点内部进行：

结点内部空间不够时，可增加一个结点存放新元素。

结点内部元素较少时，可合并相邻结点。

引用类型

引用类型是用来给一个变量取一个别名，通过该别名可以访问原来的变量：

一个引用类型的变量没有自己的内存空间，而是共享它引用的变量的内存空间。例如：

```
int x=10;
int &y=x; //y 为引用类型的变量，它是 x 的别名
y = 20; //通过 y 访问 x，效果上等价于：x = 20;
```

引用类型具有**指针类型**的一些效果。例如：

```
int *p=&x; //p 为指针类型的变量，它指向 x
*p = 30; //通过 p 访问 x，效果上等价于：x = 30
```

引用主要用于函数的参数类型，实现指针类型参数的效果，但比指针类型**抽象和安全**。

引用类型可提高参数传递的效率。例如：

```

struct A
{ int i;
  .....
};
void f(A &x) //x 使用实参 a 的内存空间，提高参数传递效率
{ .....
  ... x.i ... //访问实参
  .....
}
int main()
{ A a;
  .....
  f(a); //把 a 传给函数 f
  .....
}

```

引用类型可实现通过形参改变实参的值。例如：

```

#include <iostream>
using namespace std;
void swap(int &x, int &y) //交换两个 int 型变量的值
{   int t;
    t = x;
    x = y;
    y = t;
}
int main()
{   int a=0,b=1;
    cout << a << ',' << b << endl; //结果为： 0,1
    swap(a,b);
    cout << a << ',' << b << endl; //结果为： 1,0
    return 0;
}

```

引用类型能保证通过形参访问的永远是实参数据。

```

void f(int *p)
{ ... *p ... //通过 p 访问实参
  int m;
  p = &m; //OK
  ... *p ... //访问 m，即通过 p 可以访问实参以外的数据
}
void g(int &x)
{ ... x ... //通过 x 访问实参
  int m;
  x = &m; //Error
}

```

```

    ... x ... //通过 x 只能访问实参
}
int main()
{ int a;
  f(&a);
  g(a);
}

```

函数名重载

对于一些功能相同、参数类型或个数不同的函数，给它们取相同的名字有时会带来使用上的方便（便于理解和记忆）。函数返回值不同不能取相同的名字。

在相同的作用域中，可以用同一个名字定义多个不同的函数，这时，要求定义的这些函数应具有**不同的参数（参数个数或类型要有所不同）**。上述的做法称为函数名重载。

对重载函数调用的绑定：

确定一个对重载函数的调用对应着哪一个重载函数定义的过程称为绑定（binding，又称定联、联编、捆绑）。对重载函数调用的绑定一般是在**编译时刻**由编译程序根据实参与形参的匹配情况来决定。

绑定规则：

从形参个数与实参个数相同的重载函数中，按下面的次序选择一个：

1.精确匹配

实参与形参的**类型完全相同**，或者对实参进行“微小”的类型转换后与形参类型相同：

数组变量名->数组第一个元素的内存地址

函数名->函数的内存首地址

.....

例如，对于下面的重载函数定义：

```

void print(int);
void print(double);
void print(char);
void print(char *);

```

下面的函数调用：

```

print(1); //绑定到函数： void print(int);
print(1.0); //绑定到函数： void print(double);
print('a'); //绑定到函数： void print(char);
char s[10];
print(s); //绑定到函数： void print(char *);

```

2. 提升匹配

先对实参进行下面的类型提升，然后进行精确匹配：

按整型提升规则提升实参类型

把 float 类型实参提升到 double

把 double 类型实参提升到 long double

例如，对于下述的重载函数：

```
void print(int);
```

```
void print(double);
```

根据提升匹配，下面的函数调用：

```
print('a'); 绑定到函数：void print(int);
```

```
print(1.0f); 绑定到函数：void print(double);
```

3. 标准转换匹配

先进行下面的标准转换，然后进行精确匹配：

任何算术类型可以互相转换

枚举类型可以转换成任何算术类型

零可以转换成任何算术类型或指针类型

任何类型的指针可以转换成 void *

例如，对于下述的重载函数：

```
void print(char);
```

```
void print(char *);
```

根据标准转换匹配，下面的函数调用：

```
print(1); 绑定到函数：void print(char);
```

注：每个标准转换都是平等的，不存在哪个优先；如果存在多个标准转换后能精确匹配，则失败！（具有歧义）

4. 自定义转换匹配

5. 匹配失败

如果不存在匹配或存在多个匹配，则绑定失败。例如，对于下述的重载函数：

```
void print(char);
```

```
void print(double);
```

根据标准转换匹配，下面的函数调用将会绑定失败：

```
print(1);
```

因为 1（属于 int 型）既可以转成 char，又可以转成 double

解决办法是：

对实参进行显式类型转换，如，

```
print((char)1) 或 print((double)1)
```

增加额外的重载，如，

增加一个重载函数定义：`void print(int);`

函数指针：

C++中可以定义一个指针变量（函数指针），使其指向一个函数，可以通过这个指针变量来调用它指向的函数。例如：

```
double f(int x) { ..... } //f 是一个函数
.....
typedef double (*FP)(int);
FP fp; //fp 是一个指向函数的指针变量
或者
double (*fp)(int); //fp 是一个指向函数的指针变量
fp = &f; //fp 指向函数 f,
           //也可写成: fp = f; (编译器会做类型转换!)
(*fp)(10); //调用 fp 指向的函数 f,
           //也可写成: fp(10); (编译器会做类型转换!)
```

向函数传递函数:

函数指针主要用于通过参数把一个函数传给另一个函数。例如:

```
int func(int (*fp)(int x)) //形参 fp 为一个函数指针类型的变量
{ int i;
  .....
  ...(*fp)(i)... //或 fp(i); 调用 fp 所指向的函数 (传进来的函数)
  .....
}
int f(int x) { ..... }
int g(int x) { ..... }
int main()
{ .....
  ... func(&f) ... //或 func(f); 函数 func 中将调用 f
  ... func(&g) ... //或 func(g); 函数 func 中将调用 g
  .....
}
```

回调函数 (Callback Functions)

一个函数在执行的过程中,有时需要该函数的调用者配合做些事,这可以通过由调用者提供一个函数来实现:

原函数在执行的过程中需要调用者帮忙时,则可调用这个函数来完成。

调用者提供的这个函数称为回调函数。

回调函数通常是作为参数 (函数指针) 传给被调用者。

回调函数的应用: 能根据不同要求进行排序的排序函数

```
struct Student
{ int no;
  char name[20];
  .....
};
void sort(Student st[],int num)
{ .....
  if (st[i].no > st[j].no) //比较数组元素大小 (i<j)
```

```

        { ..... //交换 st[i]与 st[j]的位置
        }
        .....
    }

```

上述排序函数只能按 no 排序，并且是由小到大。

如果还要按 no 由大到小排序呢？

如果还要按 name 排序呢？

一般要再写若干个排序函数，这些函数的实现代码基本相同，带来重复性编程工作，并容易造成不一致！

可以给排序函数增加一个函数指针类型的参数，由调用者提供一个比较函数来指出两个元素的次序（该函数返回 true 为需要的序）：

```

void sort(Student st[],int num,
          bool (*comp)(Student *st1,Student *st2))
{ .....
    if (!comp(&st[i],&st[j]) //根据 comp 返回值来决定数组元素位置
    { ..... //交换 st[i]与 st[j]的位置
    }
    .....
}

```

例：

```

bool less_than_by_no(Student *st1,Student *st2)
{ return (st1->no <= st2->no);
}
bool greater_than_by_no(Student *st1,Student *st2)
{ return (st1->no > st2->no);
}
bool less_than_by_name(Student *st1,Student *st2)
{ return (strcmp(st1->name,st2->name)<=0);
}
bool greater_than_by_name(Student *st1,Student *st2)
{ return (strcmp(st1->name,st2->name)>0);
}
.....
Student st[100];
.....
sort(st,100, less_than_by_no); //按学号由小到大排序
sort(st,100, greater_than_by_no); //按学号由大到小排序
sort(st,100, less_than_by_name); //按姓名由小到大排序
sort(st,100, greater_than_by_name); //按姓名由大到小排序
.....

```

匿名函数——λ 表达式

对于一些临时用一下的简单函数，如果也要先给出这个函数的定义并为之取个名字，然后再通过这个函数的名字来使用它们，在有些场合下会给程序编写带来不便。C++新国际标准

(C++11) 为 C++ 提供了一种匿名函数机制——λ 表达式 (lambda expression)，利用它可以实现把函数的定义和使用合而为一。

例如，求函数 x^2 在区间 $[0,1]$ 的定积分：

```
integrate([](double x)->double { return x*x; },0,1);
```

上面红颜色部分就是一个 λ 表达式

不需要预先定义一个函数 square

λ 表达式的格式定义：

λ 表达式的常用格式为：

[<环境变量使用说明>]<形式参数><返回值类型指定><函数体>

<形式参数>：指出函数的参数及类型，其格式为：

(<形式参数表>)

如果函数没有参数，则这项可以省略。

<返回值类型指定>：指出函数的返回值类型，其格式为：

-> <返回值类型>

它可以省略，这时根据函数体中 return 返回的值隐式确定返回值类型。

<函数体>为一个复合语句。

基于范围的 for:

为了方便、可靠地实现一些简单地对一维数组元素进行遍历的操作，在 C++ 新国际标准中提供了一种基于范围的 for 语句，它能自动遍历数组元素。

例如，对一维数组所有元素求和：

```
int a[10],sum=0;
for (int n: a) sum += n; //不能改变元素的值
```

再例如，给一维数组每个元素输入一个值：

```
for (int &n: a) cin >> n; //可改变元素的值
```

类型自动判断：auto

在 C++ 新国际标准 (C++11) 中，关键词 auto 有新的含义：

定义一个变量时可以不指定它的类型，由编译器根据初始化的值自动确定它的类型。例如：

```
auto x=1+2*3.4;
```

编译程序自动确定 x 的类型为 double

再例如：

```
int f(double x,int y) { ..... }
```

```
auto fp=f;
```

编译程序自动确定 fp 的类型为：int (*fp)(double)

注意：自动生存期的局部变量不能再显式地用 auto 来指出了！